

درس سیستم عامل

تمرین عملی چهارم سوال دوم

402105727

متین باقری

402105802

فاطمیما تیمارچی

آوب)

گام های آوب را انجام دادم و خروجی به صورت زیر شد .

دلیل اینکه داخل این کانتینر خط `\[e[32m\]Timarchi-bagheri\[e[0m\]@debian:\w$` نشون داده میشه اینه که در ابتدا به ما گفته شده بود که نام `Debian user` رو باید اسم خودمون و همگروهیمون باشه و برای من اینطور نبود سر همین فایل `bashsrc` رو ادیت کردم و اسم `user` رو به صورت نمادین تغییر دادم و این خط رو به اخر فایل `bashsrc` اضافه کردم که اسم `user` اسم خودمون و همگروهیمون باشه سر همین این خط همش در کانتینر چاپ میشه و پوینت خاصی نداره :

```
Timarchi-Bagheri@debian:~/zocker$ ./zocker run --name test - container 'sh'
Running child with pid: 1
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ ps
  PID TTY          TIME CMD
 1582 ?        00:00:01 systemd
 1585 ?        00:00:00 (sd-pam)
 1605 ?        00:00:03 dbus-daemon
 1606 ?        00:00:02 pipewire
 1608 ?        00:00:00 pipewire
 1609 ?        00:00:09 wireplumber
 1610 ?        00:00:01 pipewire-pulse
 1611 ?        00:00:00 gnome-keyring-d
 1612 ?        00:00:00 mpris-proxy
 1637 ?        00:00:00 kwalletd6
 1653 ?        00:00:00 gcr-ssh-agent
 1655 ?        00:00:00 kwin_wayland_wr
 1656 ?        00:00:00 ssh-agent
 1664 ?        00:13:23 kwin_wayland
 1671 ?        00:00:02 xdg-desktop-por
 1677 ?        00:00:00 xdg-permission-
 1682 ?        00:00:00 xdg-document-po
 1731 ?        00:00:01 Xwayland
 1763 ?        00:00:00 at-spi-bus-laun
 1777 ?        00:00:00 dbus-daemon
 1784 ?        00:00:00 kmserver
 1827 ?        00:00:03 kded6
 1829 ?        00:00:00 dconf-service
 1833 ?        00:01:33 plasmashell
 1847 ?        00:00:05 kactivitymanage
 1883 ?        00:00:00 gmenudbusmenupr
 1884 ?        00:00:00 kaccess
 1885 ?        00:00:00 polkit-kde-auth
 1886 ?        00:00:07 org_kde_powerde
 1887 ?        00:00:00 xembedsnproxy
 1919 ?        00:00:00 xsettingsd
 1939 ?        00:00:00 obexd
 2025 ?        00:00:00 kdeconnectd
 2029 ?        00:00:01 xwaylandvideobr
 2055 ?        00:00:00 xdg-desktop-por
 2064 ?        00:00:03 DiscoverNotifie
 2065 ?        00:00:00 kalendarac
 2127 ?        00:00:03 xdg-desktop-por
 2467 ?        00:18:14 firefox-esr
 2472 ?        00:00:00 crashhelper
 2519 ?        00:00:02 Socket Process
 2576 ?        00:00:15 Privileged Cont
 2582 ?        00:00:00 RDP Process
 2641 ?        00:00:43 WebExtensions
 2694 ?        00:00:00 Utility Process
 2751 ?        00:00:00 plasma-browser-
 2851 ?        00:01:50 Isolated Web Co
 3279 ?        00:00:23 dolphin
 3348 ?        00:00:58 kate
 3465 ?        00:00:00 kate
```

پاسخ سوال : چون اغلب ابزار های نمایش فرایند مثل ps اطلاعات رو از درخت مجازی /proc میخوانند . آگه /proc ای که داخل کانتنینر دیده میشه در حقیقت همون /proc میزبان باشه یعنی هنوز یک mount مستقل از نوع proc داخل فضای نام PID اجرا نشده باشه اون وقت ps همون داده های فرایند میزبان رو میخونه و نتیجه مطابق انتظارمون نیست . برای اینکه ps تنها فرایند های namespace جدید رو ببینه باید فایل سیستم proc جدی رو داخل آن namespace mount مرتب سازی یا عبارتی mount کنیم تا به PID namespace جدید اشاره کنه . که در بخش ج دقیقا همین کار رو انجام دادم .

(ج)

کد تغییر یافته این قسمت رو در فایل run_container_part_J.txt قرار دادم .

پس از تغییر تابع خروجی به این صورت میشه .

```
Timarchi-Bagheri@debian:~/zocker$ make
make: 'zocker' is up to date.
Timarchi-Bagheri@debian:~/zocker$ git checkout t2
M       main.c
HEAD is now at 5abbe3f Add task 2
Timarchi-Bagheri@debian:~/zocker$ ./zocker run --name test - container 'sh'
[Child] PID namespace started, PID=1
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$ ps
  PID TTY          TIME CMD
    1 pts/1        00:00:00 sh
    2 pts/1        00:00:00 sh
    3 pts/1        00:00:00 ps
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$ sh
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$ ps
  PID TTY          TIME CMD
    1 pts/1        00:00:00 sh
    2 pts/1        00:00:00 sh
    4 pts/1        00:00:00 sh
    5 pts/1        00:00:00 ps
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$ exit
```

خب حالا تغییراتی که دادم رو توضیح میدم . در قسمت قبل فهمیدم که ps همه پردازنده های host رو نشون میده برای اینکه این رو اوکی کنم یک namespace جدید ساختم و سپس با استفاده از دستور unshare(CLONE_NEWPID | CLONE_NEWNS) که در آن CLONE_NEWPID جدید همون pid namespace ما هست و CLONE_NEWNS جدی هم برابر namespace mount جدید ما هست کاری کردم که تغییرات mount داخل کانتنینر test به host سرایت نکنه . حالا الان باید برعکسش رو هم هندل میکردم یعنی نمیگذاشتم تغییرات mount های جدید به میزبان نرن و این تغییرات mount تنها داخل کانتنینر معتبر باشن برای اینکار دستور mount(NULL, "/", NULL, MS_REC | MS_PRIVATE, NULL); را زدم . حالا در

مرحله بعدی چون میخواستیم به PID شماره 1 داخل namespace برم دستور fork رو زدم تا یک فرزند ایجاد بشه و namespace PID فقط برای فرزند اعمال بشه . در انتها دستور

mount("proc", "/proc", "proc", 0, NULL); رو اضافه کردم تا با این دستور , mount/proc مربوط به pid namespace جدید رو ساختم و در نتیجه همه چی اوکی شد و با زدن دستور ps فقط اطلاعات کانتینر نشون داده میشه که عکسش هم در بالا گذاشته بودم .

(د)

در این قسمت بعد از اینکه df زدم مطابق عکس زیر df رو پیدا نمیکرد :

```
Timarchi-Bagheri@debian:~/zocker$ df
df: cannot read table of mounted file systems: No such file or directory
Timarchi-Bagheri@debian:~/zocker$ sudo umount /proc
[sudo] password for fatima-t:
Timarchi-Bagheri@debian:~/zocker$ df
Filesystem      1K-blocks    Used Available Use% Mounted on
udev            7939180         0   7939180   0% /dev
tmpfs           1599616     2032   1597584   1% /run
/dev/nvme0n1p6  94018052 10351532  78844456 12% /
tmpfs           7998064         0   7998064   0% /dev/shm
efivarfs         192         120        68 64% /sys/firmware/efi/efivars
tmpfs           5120         12     5108   1% /run/lock
tmpfs           1024         0     1024   0% /run/credentials/systemd-journald.service
tmpfs           7998064         4   7998060   1% /tmp
/dev/nvme0n1p5  998480     8988   989492   1% /boot/efi
tmpfs           1599612     104   1599508   1% /run/user/1000
Timarchi-Bagheri@debian:~/zocker$
```

پاسخ سوال : اول اینکه علت اینکه df اجرا نمیشه اینه که یک سری mount های سیستم انگاری حذف شدن حالا علت حذف شدن اینه که لینوکس (یا Debian) برای mount ها یکسری سلسله مراتب دارن برای همین وقتی که namespace mount جدید درست میکنیم یک کپی به صورت shallow copy از namespace اصلی میگیره . باید به این اشاره کنم که پیش فرض mount propagation معمولا shared هست . برای همین اگه mount جدیدی انجام بشه مثل /proc ای که اینجا انجام دادیم اگه propagation رو تنظیم نکنیم اون وقت ممکنه این mount ای که ایجاد کردیم روی mount های دیگه که در namespace اصلی وجود ندارن اثر بزاره . به عبارت دیگه main namespace mount of namespace توسط children mounts به دلیل داشتن سلسله مراتب مخفی میشن . چون namespace جدید میاد از mount های قبلی snapshot میگیره و سپس mount های جدید روی اون اعمال میشن . حالا طبق عکس بالا دیدیم که بعد از زدن دستور \proc amount sudo همه چی درست میشه . حالا در اصل این دستور میاد

\proc ای که داخل mount namespace جدید mount شده رو unmount میکنه به عبارت دیگه میاد سلسله مراتب اصلی mount دوباره قابل دسترس میکنه . حالا اینکار رو به این صورت میکنه که میاد namespace جدید به صورت recursive mount یا propagation private میکنه و mount های قبلی hot رو از دید فرزندی که ما ساختیم پنهان میکنه و namespace host رو سالم و بدون تغییر نگه میداره بنابراین حالا df میتونه دوباره mount های جدید سیستم رو بخونه و در نتیجه df به درستی اجرا میشه .

(۵)

کد تغییر یافته این قسمت رو در فایل run_container_part_H.txt قرار دادم .

خروجی کد پس از تغییر فایل به صورت زیر میشه

```
Timarchi-Bagheri@debian:~/zocker$ make
gcc -Wall -Wextra -std=c99 -o zocker main.c
sudo setcap cap_sys_admin+ep zocker
[sudo] password for fatima-t:
Timarchi-Bagheri@debian:~/zocker$ git checkout t2
M       main.c
HEAD is now at 5abbe3f Add task 2
Timarchi-Bagheri@debian:~/zocker$ ./zocker run --name test - container 'sh'
[Child] PID namespace started, PID=1
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ ps
  PID TTY          TIME CMD
    1 pts/1        00:00:00 sh
    2 pts/1        00:00:00 sh
    3 pts/1        00:00:00 ps
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ sh
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ ps
  PID TTY          TIME CMD
    1 pts/1        00:00:00 sh
    2 pts/1        00:00:00 sh
    4 pts/1        00:00:00 sh
    5 pts/1        00:00:00 ps
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ exit
\[e[32m\]Timarchi-Bagheri\[e[0m\]@debian:\w$ exit
sh: 3: Cannot set tty process group (No such process)
[Parent] Container stopped.
Timarchi-Bagheri@debian:~/zocker$ df
Filesystem      1K-blocks    Used Available Use% Mounted on
udev             7939180         0   7939180   0% /dev
tmpfs            1599616    2040   1597576   1% /run
/dev/nvme0n1p6  94018052 10355540  78840448  12% /
tmpfs            7998064         0   7998064   0% /dev/shm
efivarfs         192         120     68      64% /sys/firmware/efi/efivars
tmpfs            5120         12     5108    1% /run/lock
tmpfs            1024         0      1024    0% /run/credentials/systemd-journald.service
tmpfs            7998064         4   7998060   1% /tmp
/dev/nvme0n1p5   998480     8988   989492   1% /boot/efi
tmpfs            1599612     108   1599504   1% /run/user/1000
Timarchi-Bagheri@debian:~/zocker$
```

حالا تغییراتی که در کد دادم رو توضیح میدم . در اینجا چون syscall mount پرچم داره و این پرچم ها نحوه propagation رو کنترل میکنن بنابراین من در کد از این پرچم ها استفاده کردم . پرچم ها چند نوع دارن که اگه

بخوام به چندتا شون اشاره کنم اول پرچم MS_SHARED هست که کارش انتشار (propagation) دو طرفه mount ها هست . پرچم بعدی MS_SLAVE هست که کارش انتشار یک طرفه از والد به فرزند هست . پرچم سوم MS_PRIVATE هست که کارش غیر فعل کردن انتشار mount هست که در قسمت من از این پرچم استفاده کردم . پرچم بعدی MS_REC هست که کارش اعمال recursive روی همه mount ها هست که م از این پرچم هم در این قسمت استفاده کردم . حالا چون پیش فرض shared هست بنابراین mount کردن proc میتونه روی mount های host یا میزبان اثر بزاره برای اینکار اومدم دستور

```
mount(NULL, "/", NULL, MS_REC | MS_PRIVATE, NULL);
```

را اضافه کردم تا همه mount های namespace جدید رو مخفی یا private کنم اما باید دقت کنم که باید این اتفاق به صورت recursive روی root اتفاق بیفته بنابراین از پرچم های MS_REC و MS_PRIVATE استفاده کردم که اولی همون طور که در بالا توضیح دادم انتشار mount ها رو متوقف میکنه و دومی هم تغییرات رو به صورت recursive روی همه mount ها اعمال میکنه . پس من الان یک mount namespace ایزوله یا private داریم پس حالا mount کردن proc فقط توی کانتینر هست و دیگه میزبان نمیدونه پس حالا دستور df به صورت صحیح اجرا میشه .

(و)

کد تغییر یافته این قسمت رو در فایل run_container_part_V.txt قرار دادم .

برای این قسمت باید فضای نام UTS رو جدا میکردم برای اینکار با استفاده از دستور

```
unshare(CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWUTS);
```

پرچم CLONE_NEWUTS رو به unshare اضافه کردم . حالا خروجی مطابق زیر میشود . که در این خروجی همون طور که مشاهده میکنید hostname سیستم فعلی که debian هست (خط آخر عکس) رو نشون میده و باید به این دقت کنیم که تغییر این تنها روی کانتینر اثر داره و در اصل یک local cop هست :

```

Timarchi-Bagheri@debian:~$ cd zocker
Timarchi-Bagheri@debian:~/zocker$ make
make: 'zocker' is up to date.
Timarchi-Bagheri@debian:~/zocker$ ./zocker run --name test - container 'sh'
[Child] PID namespace started, PID=1
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$ ps
  PID TTY          TIME CMD
    1 pts/1    00:00:00 sh
    2 pts/1    00:00:00 sh
    3 pts/1    00:00:00 ps
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$ hostname
debian
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$

```

(ن)

کد تغییر یافته این قسمت رو در فایل run_container_part_Z.txt قرار دادم .

خب تو قسمت قبل اومدیم یک UTS namespace جدید ساختیم پس اگه الان hostname رو عوض کنیم مشکلی پیش نیاد . برای اینکار مطابق صورت سوال کد های زیر رو اضافه کردم

```

if (sethostname(cfg.hostname, strlen(cfg.hostname))) != 0) {

    perror("[ERR] sethostname failed");

    exit(1);

}

```

حالا پس از اضافه کردن این خط خروجی به صورت زیر میشه . و خروجی hostname همون اسم کانتینر ما یعنی test میشه (خط آخر عکس) :

```

Timarchi-Bagheri@debian:~/zocker$ ./zocker run --name test - container 'sh'
[Child] Container started, PID=1
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$ ps
  PID TTY          TIME CMD
    1 pts/1    00:00:00 sh
    2 pts/1    00:00:00 sh
    3 pts/1    00:00:00 ps
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$ hostname
test
\\[\\e[32m\\]Timarchi-Bagheri\\[\\e[0m\\]@debian:\\w$

```


ح)

کد تغییر یافته این قسمت رو در فایل main.c قرار دادم .

در این قسمت باید دو تا namespace اضافه می‌کردیم . اولی user namespace هست که با استفاده از پرچم CLONE_NEWUSER میتونستیم UID/GID داخل کانتینر رو جدا کنیم . دومی هم Time namespace هست که با استفاده از پرچم CLONE_NEWTIME میتونستیم clock کانتینر رو از host جدا کنیم پس باید این دو تا پرچم رو به unshare اضافه می‌کردیم تا namespace ما از سیستم میزبان جدا بشه و کانتینر ایزوله بشه . در مرحله بعدی چون می‌تونیم usernamespace نیاز داره که uid/gid هاش map بشن پس باید فایل های proc/self/uid_map/ و proc/self/gid_map/ و proc/self/setgroups/ باید map بشن . پس اومدیم UID/GID داخل کانتینر رو به میزبان map کردیم . چون UID داخل کانتینر 0 هست و UID کاربر روی میزبان عددی غیر از صفره و همچنین GID داخل کانتینر برابر صفر هست اما روز میزبان غیر صفر هست و تغییر پیدا نکرده . با اینکار تونستیم کانتینر رو شبیه root کنیم اما روی میزبان امن بمونه . برای اینکه کل حرف هایی که زیم رو در کد پیاده‌سازی کنم قطعه کد زیر رو اضافه کردم (همون طور که در بالا اشاره کردم فایل کامل این قسمت در main.c هست) :

```
if (access("/proc/self/setgroups",F_OK)==0)
write_file("/proc/self/setgroups","deny");
char buf[128];
snprintf(buf,sizeof(buf),"0 %u 1\n",(unsigned)host_uid);
write_file("/proc/self/uid_map",buf);
snprintf(buf,sizeof(buf),"0 %u 1\n",(unsigned)host_gid);
write_file("/proc/self/gid_map",buf);
```

خروجی whoami در حالت عادی در ترمینال :

```
Timarchi-Bagheri@debian:~/zocker$ whoami
fatima-t
```

خروجی whoami بعد از تغییر کد (خروجی تغییر میکند و root میشود):

```
Timarchi-Bagheri@debian:~/zocker$ sudo ./zocker run --name test - container 'sh'
\[\e[32m\]Timarchi-Bagheri\[\e[0m\]@debian:\w$ whoami
root
\[\e[32m\]Timarchi-Bagheri\[\e[0m\]@debian:\w$ █
```